Table des matières

Cours 11 — Transactions et concurrence 1
Objectifs 1
Théorie 2
Points clés à retenir 9
Exercices 10

Cours 11 — Transactions et concurrence

Module 11/12 — Formation SQL avancé

Comment lire ce module — Il répond à une question que beaucoup d'analystes croient ne pas les concerner : que se passe-t-il quand plusieurs personnes écrivent en même temps. La réponse les concerne pourtant, et la section 9 dit pourquoi. Ce module n'a qu'un laboratoire, `Labo_11_OLTP_SQL.md`, parce que la concurrence se démontre là où elle se produit, c'est-à-dire sur un moteur transactionnel, et parce qu'il faut deux sessions ouvertes côte à côte pour la voir. La partie finale du labo traverse quand même sur l'entrepôt, où le problème se pose autrement.

Objectifs

À la fin de ce module, vous saurez :

- expliquer ce qu'une transaction garantit, sur un exemple où l'échec à mi-chemin serait catastrophique ;
- dire ce que chaque lettre d'ACID interdit concrètement ;
- employer `BEGIN`, `COMMIT`, `ROLLBACK` et `SAVEPOINT`, et savoir ce qui est annulable sur quel moteur ;
- nommer cinq anomalies de concurrence et reconnaître celle qui se produit devant vous ;
- choisir un niveau d'isolation en sachant ce qu'il autorise encore, et ce qu'il coûte ;
- lire un blocage et un interblocage, et dire lequel se résout tout seul ;
- expliquer le fonctionnement du MVCC de PostgreSQL, et le relier au nettoyage et au gonflement ;
- dire pourquoi la concurrence se pose autrement sur un entrepôt, et ce que le voyage dans le temps y remplace ;
- rendre un chargement rejouable, ce qui est la raison pour laquelle ce module figure dans une formation d'ingénieur analytique.

© 2026 iAlexMG. Tous droits réservés.

Théorie

1. Pourquoi une transaction

Labo : `Labo_11_OLTP_SQL.md`, sous-partie 1.1.

Prenez un virement entre deux comptes. Il s'écrit en deux commandes.

```
UPDATE compte SET solde = solde - 100 WHERE id = 1;
UPDATE compte SET solde = solde + 100 WHERE id = 2;
```

Si le moteur s'arrête entre les deux, cent dollars ont disparu. Aucune des deux commandes n'est fausse, aucune n'a échoué, et la base est pourtant dans un état qui ne devrait jamais exister.

Une transaction est la réponse à ce problème, et elle tient en une idée : un groupe de commandes qui réussit entièrement ou ne laisse aucune trace. Il n'y a pas de milieu.

```
BEGIN;
UPDATE compte SET solde = solde - 100 WHERE id = 1;
UPDATE compte SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

Le cas du virement est l'exemple canonique, et il masque l'usage le plus fréquent. Un chargement de données est exactement le même problème : vider une table puis la remplir, en deux commandes, laisse une fenêtre pendant laquelle la table est vide. Si un rapport la lit à cet instant, il ne se plaindra pas, il affichera zéro.

2. ACID, mot par mot

Labo : `Labo_11_OLTP_SQL.md`, partie 1.

Les quatre lettres se récitent souvent et se comprennent rarement. Voici ce que chacune interdit, concrètement.

Lettre	Le mot	Ce qu'elle interdit concrètement
A	Atomicité	qu'une transaction laisse la moitié de son travail derrière elle
C	Cohérence	qu'une transaction validée laisse la base en violation d'une de ses contraintes

© 2026 iAlexMG. Tous droits réservés.

Lettre	Le mot	Ce qu'elle interdit concrètement
I	Isolation	qu'une transaction voie le travail à moitié fait d'une autre
D	Durabilité	qu'une transaction validée disparaisse à la panne suivante

Deux remarques sur ces quatre lettres, parce qu'elles sont mal comprises toutes les deux.

La cohérence n'est pas une propriété que le moteur invente. Elle dit que le moteur fera respecter les contraintes que vous avez déclarées : clés, unicité, vérifications. Si vous ne déclarez rien, il n'y a rien à faire respecter, et la lettre C ne vous protège de rien. C'est la seule des quatre qui dépend entièrement de votre travail de modélisation.

L'isolation est la seule des quatre qui se règle. Les trois autres sont tout ou rien ; l'isolation offre plusieurs niveaux, et le choix est un compromis entre la sécurité et le débit. C'est le sujet des sections 4 et 5, et c'est là que se trouvent toutes les surprises.

3. Les commandes

Labo : `Labo_11_OLTP_SQL.md`, sous-parties 1.2 et 1.3.

Quatre mots suffisent.

Commande	Ce qu'elle fait
`BEGIN`	ouvre une transaction, `START TRANSACTION` en est le synonyme normalisé
`COMMIT`	valide tout ce qui précède, définitivement
`ROLLBACK`	annule tout ce qui précède, comme si rien n'avait eu lieu
`SAVEPOINT nom`	pose un point de reprise à l'intérieur de la transaction

Un point de reprise permet d'annuler une partie du travail sans tout perdre.

```
BEGIN;
INSERT INTO commande VALUES (...);
SAVEPOINT apres_entete;
INSERT INTO ligne VALUES (...);      -- échoue
ROLLBACK TO SAVEPOINT apres_entete;  -- on garde l'entête, on repart de là
INSERT INTO ligne VALUES (...);      -- corrigé
COMMIT;
```


© 2026 iAlexMG. Tous droits réservés.

Hors transaction explicite, chaque commande est sa propre transaction : c'est le mode de validation automatique, et c'est le comportement par défaut de tous les clients. Une commande isolée est donc déjà atomique, ce qui explique qu'un `UPDATE` qui touche un million de lignes ne laisse jamais la moitié du travail derrière lui.

Portabilité — Nos deux terrains savent annuler un `CREATE TABLE` ou un `ALTER TABLE` dans une transaction, ce que le Cours 03, section 8, a mesuré des deux côtés. Beaucoup de moteurs ne le savent pas et valident d'office une commande de structure, dont Oracle, MySQL et Snowflake d'après leur documentation. Ce qui compte ici, c'est la conséquence : chez vous, une migration de schéma entière peut s'exécuter dans une transaction et s'annuler en bloc si une étape échoue, et c'est un luxe qu'il ne faut pas supposer acquis ailleurs. Vous le vérifierez au laboratoire.

4. Les anomalies

Labo : `Labo_11_OLTP_SQL.md`, parties 2, 3 et 4.

Une anomalie est un comportement que la concurrence rend possible et que l'isolation cherche à empêcher. Il y en a cinq à connaître, et la dernière surprend même les gens expérimentés.

Anomalie	Ce qui se passe	Exemple en une phrase
Lecture sale	on lit une donnée qu'une autre transaction n'a pas encore validée	on lit un solde qui va être annulé
Lecture non répétable	on lit deux fois la même ligne et elle a changé entre les deux	le total du rapport ne correspond plus à son détail
Ligne fantôme	on relance la même requête et de nouvelles lignes sont apparues	le compte des commandes du jour augmente en cours de calcul
Mise à jour perdue	deux transactions lisent, calculent, écrivent, et la seconde écrase la première	deux retraits de 30 $ et de 50 $ laissent un seul retrait
Anomalie d'écriture	chacune vérifie une règle, chacune écrit, et la règle est violée à deux	deux médecins de garde se retirent en même temps, il n'en reste aucun

Les trois premières sont des anomalies de lecture, et les niveaux d'isolation les traitent directement. Les deux dernières sont plus vicieuses, parce que chaque transaction prise isolément est correcte. C'est leur combinaison qui est fausse, et aucune ne fait rien d'illégal.

L'anomalie d'écriture, qu'on appelle aussi *write skew*, mérite qu'on s'y arrête. Deux médecins sont de garde, et le règlement exige qu'il en reste au moins un. Chacun ouvre une transaction, compte les médecins de garde, en trouve deux, conclut qu'il peut se retirer, et se retire. Les deux transactions valident. Il ne reste personne, et aucune contrainte de la base n'a été violée, parce que la règle porte sur une propriété collective que le moteur ne connaît pas.

5. Les niveaux d'isolation

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_11_OLTP_SQL.md`, sous-parties 2.3 et 4.2.

La norme SQL définit quatre niveaux, du plus permissif au plus strict. Ce que chacun autorise encore est le seul tableau à retenir de cette section.

Niveau	Lecture sale	Lecture non répétable	Ligne fantôme	Mise à jour perdue	Anomalie d'écriture
Lecture non validée	possible selon la norme	possible	possible	possible	possible
Lecture validée	impossible	possible	possible	possible	possible
Lecture répétable	impossible	impossible	possible selon la norme	impossible sur PostgreSQL	possible
Sérialisable	impossible	impossible	impossible	impossible	impossible

La colonne de la mise à jour perdue mérite un mot, parce qu'elle explique ce que la lecture répétable apporte vraiment. PostgreSQL ne se contente pas d'y figer votre vue des données : si vous modifiez une ligne qu'une autre transaction a modifiée depuis le début de la vôtre, il **refuse l'écriture** plutôt que d'écraser. Mesuré, avec ce message : « n'a pas pu sérialiser un accès à cause d'une mise à jour en parallèle », et la transaction est annulée. La mise à jour n'est donc pas perdue, elle est signalée, et c'est à votre application de rejouer. Une contrainte de plus à gérer, et une famille de bogues silencieux en moins.

Une précision sur le moment, parce qu'elle surprend souvent : l'instantané d'une transaction en lecture répétable est pris à sa **première requête**, pas au `BEGIN`. Une transaction ouverte puis laissée en attente cinq minutes ne fige rien pendant ces cinq minutes.

Trois précisions sur ce que fait PostgreSQL, parce qu'il est plus strict que la norme sur deux points.

Il ne permet jamais la lecture sale, même si vous demandez le niveau le plus permissif : la demande est acceptée et traitée comme une lecture validée. Son niveau par défaut est la lecture validée, ce qui veut dire que chaque commande voit un instantané pris à son propre démarrage, et que deux commandes d'une même transaction peuvent donc voir deux états différents.

Son niveau de lecture répétable est en réalité un instantané complet pris à l'ouverture de la transaction, et il empêche donc aussi les lignes fantômes, ce que la norme n'exige pas. Il laisse passer l'anomalie d'écriture.

Son niveau sérialisable ajoute une détection des dépendances entre transactions. Il ne bloque pas davantage ; il laisse les transactions travailler et en annule une quand leur entrelacement ne correspond à aucun ordre séquentiel possible. Autrement dit, il déplace le coût vers un échec qu'il faut savoir rattraper.

À retenir — Un niveau sérialisable oblige votre code à savoir recommencer. Le moteur annule une transaction sur deux dans un conflit, avec un code d'erreur précis, et il attend que l'application la rejoue. Choisir ce niveau sans écrire la reprise revient à choisir des erreurs plutôt que des anomalies.

6. Les verrous

Labo : `Labo_11_OLTP_SQL.md`, sous-parties 1.3, 5.1 et 5.2.

© 2026 iAlexMG. Tous droits réservés.

Un verrou est le mécanisme par lequel le moteur empêche deux écritures de se marcher dessus. Deux distinctions suffisent, plus une mise au point qui doit venir avant les deux.

La mise au point, parce que beaucoup de cours l'escamotent : **sur PostgreSQL, un `SELECT` ordinaire ne pose aucun verrou de ligne**. C'est mesurable, et le labo vous le fera mesurer : une transaction qui lit une table n'a aucune entrée dans `pg_locks` au niveau de la ligne, seulement un verrou léger sur la relation, celui qui empêche qu'on la supprime sous ses pieds. Une lecture n'attend donc jamais une écriture, et une écriture n'attend jamais une lecture. C'est le MVCC de la section 7 qui rend cela possible, et c'est ce qui distingue PostgreSQL des moteurs plus anciens où toute lecture posait un verrou partagé que l'écriture devait attendre. Si vous avez appris la concurrence sur l'un de ceux-là, c'est la première chose à désapprendre.

Cela dit, les verrous existent bel et bien, entre écritures. Un verrou partagé peut être tenu par plusieurs transactions à la fois ; un verrou exclusif n'est tenu que par une seule. Un exclusif attend que les partagés soient relâchés, et réciproquement.

Un verrou de ligne ne bloque qu'une ligne, un verrou de table bloque la table entière. Le premier est la règle sur une écriture ordinaire ; le second apparaît sur les commandes de structure, et c'est ce qui rend un `ALTER TABLE` dangereux en pleine journée, même quand il est instantané en apparence : il attend son verrou, et pendant qu'il attend, il fait attendre tout le monde derrière lui.

Une commande mérite d'être nommée à part, parce qu'elle revient souvent dans les chargements : `TRUNCATE` prend le verrou le plus fort qui existe, l'exclusif d'accès, mesuré. Contrairement à un `DELETE`, il bloque donc aussi les **lectures** jusqu'à la fin de la transaction. Un `TRUNCATE` suivi d'un `INSERT` dans une transaction ne protège pas les lecteurs, il les fait attendre, ce qui n'est pas du tout la même chose.

Un blocage n'est pas une erreur. Une transaction qui attend un verrou attend, indéfiniment s'il le faut, et elle repart dès que le verrou se libère. C'est le comportement normal, et c'est aussi l'explication de la plupart des applications qui semblent figées.

L'interblocage, lui, est une impasse. La transaction A tient le verrou 1 et demande le 2 ; la transaction B tient le 2 et demande le 1. Aucune ne peut avancer, et aucune ne relâchera son verrou. Le moteur le détecte, choisit une victime, l'annule avec un message explicite, et laisse l'autre continuer. Un interblocage se résout donc tout seul, à condition que l'application sache rejouer la transaction annulée.

La règle qui les évite tient en une phrase : verrouillez toujours les ressources dans le même ordre. La plupart des interblocages viennent de deux traitements qui parcourent les mêmes tables en sens inverse.

7. Le MVCC de PostgreSQL

Labo : `Labo_11_OLTP_SQL.md`, partie 6.

Voici le mécanisme qui explique la moitié des comportements de PostgreSQL, y compris ceux que le Cours 10 a laissés en suspens.

MVCC veut dire *multiversion concurrency control*, le contrôle de concurrence multiversion : plusieurs versions d'une même ligne coexistent, et chaque transaction voit celle qui lui revient.

PostgreSQL ne modifie jamais une ligne sur place. Un `UPDATE` écrit une nouvelle version de la ligne ailleurs et marque l'ancienne comme périmée à partir d'un certain numéro de transaction. Un `DELETE` ne supprime rien, il marque. Chaque transaction porte un numéro, et chaque version de ligne porte les numéros qui décident de sa visibilité.

© 2026 iAlexMG. Tous droits réservés.

Trois conséquences en découlent, et elles sont toutes visibles.

Les lecteurs ne bloquent pas les écrivains, et réciproquement. Un rapport qui lit une table pendant qu'on l'écrit voit l'état d'avant, sans attendre et sans gêner personne. C'est la propriété la plus précieuse du modèle, et c'est ce qui rend PostgreSQL confortable pour un analyste.

Les versions périmées s'accumulent. Une table très mise à jour grossit alors que son nombre de lignes ne change pas, et ses parcours ralentissent en proportion. Le nettoyage, `VACUUM`, récupère cet espace, et un démon le lance tout seul ; il arrive qu'il n'arrive pas à suivre, et c'est l'une des causes classiques d'une base qui se dégrade lentement.

Et le parcours d'index seul du Cours 10 s'explique enfin. Un index ne porte pas l'information de visibilité : le moteur qui trouve une entrée d'index ne sait pas si la ligne correspondante est visible pour votre transaction. Il doit donc ouvrir la table pour vérifier, sauf si la page est marquée comme entièrement visible, ce que fait le nettoyage. Voilà pourquoi un `Index Only Scan` affiche des accès à la table tant qu'un `VACUUM` n'est pas passé.

8. Côté entrepôt

Labo : `Labo_11_OLTP_SQL.md`, partie 8.

La concurrence se pose autrement sur un entrepôt, et il faut comprendre pourquoi avant de transposer quoi que ce soit.

Personne n'y met à jour une ligne à la fois. Les écritures arrivent par lots : un chargement, une reconstruction de table, une fusion de plusieurs milliers de lignes. Il n'y a donc presque jamais deux transactions qui se disputent la même ligne, et tout le problème du chapitre précédent s'évapore.

Notre entrepôt pousse ce raisonnement à son terme, et il faut le voir pour comprendre le genre de compromis qu'un moteur analytique s'autorise. DuckDB est un moteur intégré : le fichier de la base n'appartient qu'à un processus à la fois. Ouvrez l'entrepôt depuis une ligne de commande, lancez une écriture qui dure, et toute autre tentative d'ouverture échoue immédiatement, même en lecture seule :

```
IO Error: Cannot open file "adventureworksdw.duckdb": Le processus ne peut pas accéder au fichier
car ce fichier est utilisé par un autre processus.
File is already open in … duckdb.exe (PID 14064)
```

Le message est net, il nomme même le processus fautif, et c'est une chance : ce genre de conflit se diagnostique en une seconde. Mais mesurez ce que cela veut dire. Il n'y a pas d'isolation à régler, pas de verrou de ligne, pas de MVCC, pas de lecture concurrente pendant une écriture : il y a un écrivain, et c'est tout. Les quatre niveaux d'isolation de la section 3 n'ont simplement pas d'objet.

Cette contrainte est acceptable parce qu'elle correspond à l'usage : un entrepôt local sert à analyser, et l'alimentation s'y fait par lots, pas en continu depuis vingt applications. Les grands entrepôts lèvent la contrainte sans changer le principe. Snowflake fonctionne par instantané, d'après sa documentation : chaque commande voit l'état validé au moment où elle commence, une lecture n'est jamais bloquée par une écriture, un seul niveau d'isolation existe, et deux fusions concurrentes sur la même table se sérialisent.

Les transactions, en revanche, existent bel et bien sur notre entrepôt, et elles font exactement ce qu'on attend. Un `DELETE` suivi d'un `ROLLBACK` rend ses mille lignes à la table, mesuré ; et le DDL s'y annule aussi, contrairement à ce que font la plupart des entrepôts. Sur ce point précis, nos deux terrains sont ensemble du côté rare, comme le disait déjà le Cours 03, section 8.

© 2026 iAlexMG. Tous droits réservés.

Reste ce que notre entrepôt n'a pas et que les grandes plateformes offrent : le voyage dans le temps. Une table y conserve ses états passés pendant une durée configurable, et on interroge l'état d'il y a une heure, ou celui d'avant une commande précise, en ajoutant une clause à la requête ; une table supprimée par erreur se récupère par `UNDROP TABLE`, d'après la documentation de Snowflake. Sur un fichier local, l'équivalent du filet est plus rustique et tout aussi efficace : on copie le fichier de l'entrepôt avant une opération risquée, ce qui prend une seconde sur les 25 Mo du nôtre.

Dans les deux cas, le filet ne remplace pas la transaction, et c'est ce qu'il faut retenir. Il répare après coup, sur décision humaine ; la transaction empêche l'état intermédiaire d'exister. Un rapport lancé pendant les trois secondes où la table était vide aura quand même affiché zéro.

9. Ce que ça change pour un ingénieur analytique

Labo : `Labo_11_OLTP_SQL.md`, partie 7.

Voici la section qui justifie le module, et le pont vers le capstone.

Un chargement doit être rejouable. Le vôtre le sera un jour, parce qu'il aura échoué à mi-chemin, parce qu'un fichier sera arrivé deux fois, ou parce que quelqu'un le relancera par prudence. Un chargement rejouable est un chargement qui, lancé deux fois avec la même entrée, laisse la base dans le même état que s'il avait été lancé une fois. On dit qu'il est idempotent, et ce n'est pas une qualité optionnelle.

Trois motifs le garantissent, du plus grossier au plus fin.

Motif	Comment	Quand il convient
Remplacement complet	reconstruire la table entière, dans une transaction	table petite ou moyenne, source complète disponible
Suppression et réinsertion par tranche	supprimer la période concernée, puis la réinsérer, dans une transaction	chargement par jour ou par mois, source partielle
Fusion par clé	`MERGE`, qui met à jour ce qui existe et insère le reste	source par changements, clé métier fiable

Les trois ont besoin de la transaction pour la même raison. Sans elle, il existe un instant où la table est vide ou incomplète, et un lecteur peut tomber dessus. Avec elle, aucun lecteur ne voit jamais autre chose que l'état d'avant ou l'état d'après.

Le `MERGE`, dont la syntaxe est présentée au Cours 03, section 6, fait tenir les deux besoins ensemble : il met à jour ce qui existe et insère ce qui manque, en une seule commande. Il est idempotent par construction, à une condition qui n'est pas anodine : la clé doit être unique dans la source. Une source qui contient deux fois la même clé fait échouer la commande sur certains moteurs, et produit un résultat non déterministe sur d'autres. Le contrôle d'unicité de la source fait donc partie du chargement, pas des bonnes intentions.

La seconde raison qui vous concerne est plus discrète. Une lecture qui porte sur plusieurs tables doit voir un état cohérent. Si votre rapport lit les commandes, puis les lignes de commande, et qu'un chargement passe entre les deux, votre total et votre détail ne coïncideront pas, et vous chercherez l'erreur dans votre SQL.

© 2026 iAlexMG. Tous droits réservés.

La parade dépend du terrain, et il faut le dire, parce que c'est sur l'entrepôt que vous lirez la plupart de vos rapports. Sur PostgreSQL, ouvrir une transaction en lecture répétable autour des deux requêtes règle le problème : elles voient le même instantané, et presque personne ne le fait. Sur notre entrepôt, la question ne se pose pas, faute d'un second écrivain pour changer les données sous vos pieds. Sur un grand entrepôt, elle se pose et la parade ne s'y transpose pas telle quelle : un seul niveau d'isolation y existe, la lecture validée, d'après la documentation de Snowflake, et deux `SELECT` d'une même transaction peuvent donc encore se contredire. La réponse y est le voyage dans le temps, en fixant les deux requêtes au même instant passé.

Retenez la question plutôt que la recette : par quel moyen mon moteur me garantit-il que deux requêtes voient le même état ? Il y a toujours une réponse, elle n'a pas le même nom partout, et elle n'est jamais le réglage par défaut.

Points clés à retenir

- Une transaction est un groupe de commandes qui réussit entièrement ou ne laisse aucune trace.
- Un chargement en deux temps, vidage puis remplissage, laisse une fenêtre où la table est vide. Un lecteur qui tombe dedans affichera zéro sans se plaindre.
- La cohérence d'ACID ne fait respecter que les contraintes que vous avez déclarées. Sans contraintes, elle ne protège de rien.
- L'isolation est la seule des quatre lettres qui se règle, et c'est là que se trouvent toutes les surprises.
- `SAVEPOINT` permet d'annuler une partie du travail sans perdre le reste.
- Le DDL s'annule dans une transaction sur nos deux terrains, ce qui est rare : Oracle, MySQL et Snowflake le valident d'office, d'après leur documentation. Une migration de schéma peut donc s'exécuter en bloc chez vous, et pas forcément ailleurs.
- Cinq anomalies : lecture sale, lecture non répétable, ligne fantôme, mise à jour perdue, anomalie d'écriture. Les deux dernières sont faites de transactions individuellement correctes.
- PostgreSQL ne permet jamais la lecture sale. Son défaut est la lecture validée, où chaque commande voit son propre instantané.
- La lecture répétable de PostgreSQL empêche aussi les lignes fantômes, plus que la norme n'exige. Elle laisse passer l'anomalie d'écriture.
- Le niveau sérialisable n'empêche pas, il annule. Le choisir sans écrire la reprise revient à choisir des erreurs plutôt que des anomalies.
- Un blocage est normal et se résout quand le verrou se libère. Un interblocage est une impasse que le moteur casse en annulant une victime.
- On évite les interblocages en verrouillant toujours les ressources dans le même ordre.
- PostgreSQL ne modifie jamais une ligne sur place. Il écrit une nouvelle version et marque l'ancienne, d'où le nettoyage, le gonflement, et le `Heap Fetches` du Cours 10.

© 2026 iAlexMG. Tous droits réservés.

- Sous MVCC, les lecteurs ne bloquent pas les écrivains. C'est ce qui rend un rapport long inoffensif.
- Sur l'entrepôt, les écritures arrivent par lots et la concurrence ligne à ligne disparaît. Le voyage dans le temps répare après coup ; il ne remplace pas la transaction.
- Un chargement doit être idempotent : lancé deux fois, il laisse le même état qu'une fois. Trois motifs le garantissent, et les trois ont besoin d'une transaction.
- Le `MERGE` est idempotent à une condition : la clé doit être unique dans la source, et c'est à vous de le vérifier.
- Une lecture sur plusieurs tables doit voir un état cohérent. Une transaction en lecture répétable autour du rapport le garantit, et presque personne ne le fait.

Exercices

Les exercices de ce module et leur corrigé vivent dans `Exercices_corriges_11_SQL.md`. Faites-les après le laboratoire : plusieurs énoncés supposent que vous avez provoqué un interblocage de vos propres mains.

© 2026 iAlexMG. Tous droits réservés.